- **Do NOT use the orthotropic card.** Applying it on top of meshed holes **double-counts** the softening: the geometry already provides the hole-induced compliance, and the card would soften it a second time.
- **Stress around the holes only exists if the holes are meshed.** A homogenized block has no holes, so hole stress is undefined in it. Your goal (hole stress) *requires* the detailed mesh, which *requires* isotropic material. The two choices are locked together.

```
*MATERIAL, NAME=INCONEL_650F
*ELASTIC
27.0E6, 0.3
```

JOB 1 — Generate the substructure

14.1 Reference nodes and coupling

Two interfaces, each lumped to one reference node:

- `RP_BOT` — centroid of the bottom face
- `RP_TOP` — centroid of the top face

```
*NODE, NSET=RP_BOT
999001, <x_c>, <y_c>, 0.0
*NODE, NSET=RP_TOP
999002, <x_c>, <y_c>, 2.985
```

Define the face node sets (`SURF_BOT`, `SURF_TOP`) and couple each to its RP. **Kinematic** coupling, all 6 DOF:

```
*COUPLING, CONSTRAINT NAME=CPL_BOT, REF NODE=RP_BOT, SURFACE=SURF_BOT
*KINEMATIC
1, 6
*COUPLING, CONSTRAINT NAME=CPL_TOP, REF NODE=RP_TOP, SURFACE=SURF_TOP
*KINEMATIC
1, 6
```

> **Why kinematic:** for a pull test the grips are effectively rigid, so forcing each end face to move as a rigid plane matches the physics. (If you ever mate this to a *compliant* part, revisit — see §2.2.)

The key point: stress recovery must be requested **at generation time**. Once the substructure is generated without it, the interior DOF are condensed out and the hole stresses are **gone permanently** — you cannot recover them later.

```
*STEP, NAME=GEN
*SUBSTRUCTURE GENERATE, TYPE=Z101, OVERWRITE,
      RECOVERY MATRIX=YES, MASS MATRIX=NO
*RETAINED NODAL DOFS, SORTED=YES
RP_BOT, 1, 6
RP_TOP, 1, 6
**
** --- Recovery: keep interior stress/strain around the holes ---
*ELEMENT RECOVERY MATRIX, ELSET=ELS_HOLES
S, E
**
** --- Write the stiffness matrix to a readable file ---
*SUBSTRUCTURE MATRIX OUTPUT, STIFFNESS=YES, FILE NAME=perf_K, OUTPUT
FILE=USER DEFINED
*END STEP
```

Notes:

- `TYPE=Z101` — your superelement's ID. Any `Zxxx` label.
- `*RETAINED NODAL DOFS` — the 12 DOF that survive condensation.
- `ELSET=ELS_HOLES` — define this element set to contain the elements **ringing the holes** (the ones whose stress you care about). You *can* pass the whole part, but the recovery matrix cost scales with the element count, so restrict it to the hole rings if the model is large.
- `MASS MATRIX=NO` — static analysis only.

Run as **Job 1** (e.g. `perf_gen`). Outputs: `perf_gen.sim` / `.sup` (the superelement) and `perf_K.mtx` (the stiffness matrix).

JOB 2 — Example analysis: pull the substructure

A separate model that *uses* the superelement — this is where the BCs, step, and load live.

14.3 Assemble the superelement

```
*HEADING
Pull test of perforated-region superelement
**
*PART, NAME=PERF
*END PART
**
*ASSEMBLY, NAME=ASSY
*INSTANCE, NAME=SE, LIBRARY=perf_gen
*END INSTANCE
**
** Superelement as a single element, connecting its two retained nodes:
*ELEMENT, TYPE=Z101, ELSET=ESUB
1, 999001, 999002
*SUBSTRUCTURE PROPERTY, ELSET=ESUB
*END ASSEMBLY
```

*(In CAE: **Model** → **Substructure** → **Import**, then place it in the assembly. The `.inp` route above is usually faster and less fussy.)*

14.4 Step

```
*STEP, NAME=PULL, NLGEOM=NO
*STATIC
1.0, 1.0, 1.0, 1.0
```

Linear elastic, one increment. NLGEOM off — a substructure is inherently linear.

14.5 Boundary conditions

Fix the bottom reference node completely; this grounds all rigid-body modes:

```
*BOUNDARY
999001, 1, 6, 0.0        ** RP_BOT fully fixed
```

14.6 Load — two options

Option A — prescribed displacement (recommended). Gives a clean reaction force to read, and the axial stiffness falls right out.

```
*BOUNDARY
999002, 3, 3, 0.002985   ** RP_TOP: U3 = delta (0.1% of t = 2.985)
999002, 1, 2, 0.0        ** no lateral drift
999002, 4, 6, 0.0        ** no rotation (rigid grip)
```

Then `K_axial = RF3(RP_TOP) / δ`.

Option B — applied force.

```
*CLOAD
999002, 3, 10000.0       ** 10 kip in Z
```

Then `K_axial = F / U3(RP_TOP)`.

> Use **Option A** for a stiffness check (well-conditioned, one reaction to read). Use **Option B** if you want to see the deflection under a known service load. To **push** instead of pull, flip the sign.

14.7 Output requests — including the recovered hole stress

```
*OUTPUT, FIELD
*NODE OUTPUT
U, RF
**
** --- Recover interior stress around the holes ---
*SUBSTRUCTURE RECOVERY, ELSET=ESUB
*ELEMENT OUTPUT, ELSET=ELS_HOLES
S, E, MISES
**
*OUTPUT, HISTORY
*NODE OUTPUT, NSET=RP_TOP
U3, RF3
*END STEP
```

`*SUBSTRUCTURE RECOVERY` is what pulls the interior solution back out using the recovery matrix from Job 1. Without the Job-1 request, this does nothing.

Run as **Job 2** (e.g. `perf_pull`). In Visualization you will see the recovered stress contours **around the holes**, plus the interface reactions.

14.8 Verify the result

1. **Read** `RF3` **at** `RP_TOP` → `K_axial = RF3 / δ`.

2. **Hand-check:** `K_axial ≈ E_m · A_net / t`, where `A_net` = the *net* (solid) cross-sectional area of the perforated region — i.e. gross area × solid fraction. For the §10 array, solid fraction = **0.683**, so `A_net = 0.683 · A_gross` and

```
K_axial ≈ 0.683 · E_m · A_gross / 2.985
```

Expect the FE value to land **slightly below** this (the ideal prismatic calc ignores local compliance around the holes). Agreement within a few percent = the substructure is behaving. 3. **Hole stress sanity:** peak Mises at the hole edge should exceed the net-section average by a stress-concentration factor of roughly **2–3** for a perforated array. If the recovered stress is uniform or shows no concentration, the recovery matrix did not work — check the Job-1 `*ELEMENT RECOVERY MATRIX` request.

14.9 Reading the stiffness matrix Abaqus generates (your question)

14.9.1 Make Abaqus write it

The `*SUBSTRUCTURE MATRIX OUTPUT` line in §14.2 does it:

```
*SUBSTRUCTURE MATRIX OUTPUT, STIFFNESS=YES, FILE NAME=perf_K, OUTPUT
FILE=USER DEFINED
```

- `OUTPUT FILE=USER DEFINED` → writes **`perf_K.mtx`**, an ASCII text file you can open and parse. *(The default, `OUTPUT FILE=RESULTS FILE`, writes to the binary `.fil` — harder to read. Use USER DEFINED.)*
- To also get mass: `MASS=YES`.

14.9.2 What the file looks like

`perf_K.mtx` contains the **lower-triangular** entries of the symmetric reduced stiffness, in the `*MATRIX INPUT` / `*USER ELEMENT` format:

```
** rows: node, DOF, node, DOF, value
999001, 1, 999001, 1,  1.234e+07
999001, 2, 999001, 1, -5.678e+05
999001, 2, 999001, 2,  2.345e+07
...
```

Each line is `node_i, dof_i, node_j, dof_j, K_ij`. With 12 retained DOF, the full matrix is **12 × 12** and the file holds its 78 lower-triangular terms.

`*RETAINED NODAL DOFS, SORTED=YES` orders them ascending by node then DOF:

Index	Node	DOF
1–6	`RP_BOT` (999001)	$u_1\ u_2\ u_3\ \theta_1\ \theta_2\ \theta_3$
7–12	`RP_TOP` (999002)	$u_1\ u_2\ u_3\ \theta_1\ \theta_2\ \theta_3$

So `K(9,9)` is the axial (Z–Z) stiffness at `RP_TOP` — the term to compare against §14.8's hand calc.

14.9.4 MATLAB — parse and inspect

matlab

```
% read_substructure_K.m
% Parse Abaqus *SUBSTRUCTURE MATRIX OUTPUT (.mtx, USER DEFINED) into a
% full symmetric stiffness matrix, then run sanity checks.
% Line format: node_i, dof_i, node_j, dof_j, value   (lower triangle)

clear; clc;

fname = 'perf_K.mtx';
nodes = [999001, 999002];    % retained nodes, in SORTED order
ndof  = 6;                   % DOF per node
N     = numel(nodes)*ndof;   % = 12

% Map (node, dof) -> global index
idx = containers.Map('KeyType','char','ValueType','double');
for a = 1:numel(nodes)
    for d = 1:ndof
        idx(sprintf('%d_%d', nodes(a), d)) = (a-1)*ndof + d;
    end
end

K = zeros(N);
fid = fopen(fname,'r');
while ~feof(fid)
    ln = strtrim(fgetl(fid));
    if isempty(ln) || startsWith(ln,'*') || startsWith(ln,'**'), continue; end
    v = sscanf(strrep(ln,',',' '), '%f');
    if numel(v) < 5, continue; end
    i = idx(sprintf('%d_%d', v(1), v(2)));
    j = idx(sprintf('%d_%d', v(3), v(4)));
    K(i,j) = v(5);
    K(j,i) = v(5);           % mirror (file is lower-triangular)
end
fclose(fid);

fprintf('Reduced stiffness: %d x %d\n\n', N, N);
disp(K);

%% --- Sanity checks ------------------------------------------------
asym = max(abs(K - K.'),[],'all') / max(abs(K),[],'all');
fprintf('Asymmetry            = %.2e   (want ~0)\n', asym);
fprintf('Condition number     = %.3e\n', cond(K));

ev = sort(eig(K));
```

```
nrb = sum(abs(ev) < 1e-8*max(abs(ev)));
fprintf('Near-zero eigenvalues = %d  (expect 6: free-free rigid-body modes)\n',
% The generated superelement is UNCONSTRAINED -> 6 rigid-body modes are correct
% They vanish once RP_BOT is fixed in Job 2.

%% --- Axial term ----------------------------------------------------
K_axial = K(9,9);            % RP_TOP, DOF 3 (Z)
fprintf('\nK(9,9) axial (RP_TOP, Z) = %.4e lb/in\n', K_axial);

% Hand check: K ~ solid_frac * E_m * A_gross / t
E_m        = 27e6;
solid_frac = 0.683;          % sec 10.3 Heron area fraction
A_gross    = 199.82;         % <-- SET to YOUR region's gross area (in^2)
t          = 2.985;
K_hand = solid_frac * E_m * A_gross / t;
fprintf('Hand calc  = %.4e lb/in\n', K_hand);
fprintf('Difference = %+.1f%%  (FE should sit slightly BELOW hand calc)\n', ...
        100*(K_axial - K_hand)/K_hand);
```

> **Expect 6 near-zero eigenvalues** when you check the raw generated matrix — the superelement is unconstrained (free-free), so six rigid-body modes are *correct*, not a bug. They disappear once `RP_BOT` is fixed in Job 2.

14.10 Pitfalls

- **Forgetting `RECOVERY MATRIX=YES` at generation.** The hole stress is then unrecoverable — you must regenerate. This is the #1 error for your use case.
- **Using the §9 orthotropic card with meshed holes** → double-counts the softening (§14.0).
- `OUTPUT FILE=RESULTS FILE` → binary `.fil`, awkward to read. Use `USER DEFINED` for ASCII.
- **Recovery matrix over the whole part** when only hole rings matter → large file, slow generation. Scope `ELSET=ELS_HOLES`.
- **Expecting a positive-definite raw matrix.** The generated superelement is free-free; 6 zero eigenvalues are expected.
- **Forgetting `*SUBSTRUCTURE RECOVERY` in Job 2** → the recovery matrix exists but is never invoked, so no interior stress appears.

14.11 GUI (Abaqus CAE) Walkthrough

Honest caveat up front: CAE's substructure support is **partial**. Generation and usage exist under **Model → Substructure**, but two things have **no GUI equivalent** and must be added via the **Keyword Editor**:

- `*ELEMENT RECOVERY MATRIX` (scoping recovery to the hole elements)
- `*SUBSTRUCTURE MATRIX OUTPUT` (writing the readable `.mtx`)

Both are flagged **[KEYWORD EDITOR]** below. Everything else is clickable.

14.11.0 Finding the centroid of the hexagonal tubesheet

You need this for `RP_BOT` / `RP_TOP`. **Do not eyeball it.**

Method 1 — query the mass properties (best, exact):

1. Switch to the **Part** module, with the tubesheet part displayed.
2. **Tools → Query…**
3. Select **Mass properties** (under General Queries).
4. Pick the solid part → **Done**.
5. The message area prints **Volume**, **Mass**, and **Centroid (X, Y, Z)**. Read the X, Y directly.

> This is the true area/volume centroid of the *actual* hexagon, including the angled sides — which is exactly what you want and what no hand sketch will give you reliably.

Method 2 — probe the face centroid:

1. **Tools → Query → Probe values**.
2. Set **Probe: Elements** (or Nodes), pick the face of interest.
3. Less direct than Method 1; use it as a spot check.

Sanity check by hand: your hexagon has a flat bottom, two vertical sides, two inward-angled sides, and a flat top — so it is **symmetric about its vertical centerline**. Therefore **X_centroid = the midpoint of the flat bottom edge** (by symmetry). Only **Y_centroid** requires the query. If Method 1's X differs from your symmetry axis, something is off in the geometry.

Using it: the two reference points share the same (X, Y) = the centroid, and differ only in Z:

```
RP_BOT = (X_c, Y_c, 0.0)
RP_TOP = (X_c, Y_c, 2.985)
```

> **Note:** the centroid is the natural, unbiased choice for the reference point, but with **kinematic coupling** the RP location does **not** change the physics — the face is rigid, so any RP on it gives an equivalent (just differently-referenced) stiffness. The centroid keeps the moment terms clean and interpretable, which is why we use it.

JOB 1 (GUI) — Generate the substructure

Step 1 — Property module: isotropic material

1. **Module: Property**
2. **Material → Create.** Name: `INCONEL_650F`.
3. **Mechanical → Elasticity → Elastic.** Type: **Isotropic.**
4. Young's Modulus = `27e6`, Poisson's Ratio = `0.3`. **OK.**
   - *(§14.0: isotropic — **not** the §9 orthotropic card, because the holes are meshed.)*
5. **Section → Create.** Solid → Homogeneous. Material: `INCONEL_650F`. **OK.**
6. **Assign → Section.** Select the whole part → **Done** → **OK.**

Step 2 — Assembly module: instance + reference points

1. **Module: Assembly** → **Instance → Create** → select the part → **Independent (mesh on instance)** → **OK.**
2. Get the centroid (§14.11.0).
3. **Tools → Reference Point.** Enter coordinates `(X_c, Y_c, 0.0)` → this is **RP-1** (`RP_BOT`).
4. **Tools → Reference Point** again: `(X_c, Y_c, 2.985)` → **RP-2** (`RP_TOP`).
5. **Tools → Set → Create** → Type: **Geometry** → name `RP_BOT_SET`, select RP-1 → **Done.** Repeat for `RP_TOP_SET`.

Step 3 — Define the interface surfaces

1. **Tools → Surface → Create.** Name: `SURF_BOT`. Type: **Geometry.**
2. Select the **bottom face** (Z = 0) of the perforated region → **Done.**
3. Repeat for `SURF_TOP` (the Z = 2.985 face).
4. **Tools → Set → Create** → Type: **Element** → name `ELS_HOLES` → select the elements **ringing the holes** (the ones whose stress you want to recover).
   - *Tip:* easiest via **Tools → Set → Create → Element**, then use **by angle / by feature** or box-select the hole rings. Keep this set as small as is useful — the recovery matrix cost scales with it.

Step 4 — Interaction module: kinematic coupling

1. **Module: Interaction**
2. **Constraint → Create → Coupling** → Continue.
3. **Control point:** pick **RP-1** (`RP_BOT`) → **Done.**
4. **Surface:** pick `SURF_BOT` → **Done.**
5. In the dialog: **Coupling type = Kinematic**. Check **U1 U2 U3 UR1 UR2 UR3** (all six). **OK.**
6. **Repeat** for **RP-2** / `SURF_TOP`.

> Why kinematic: rigid grips for a pull test (§14.1). If you later mate this to a compliant part, revisit §2.2.

Step 5 — Step module: substructure generation

1. **Module: Step**
2. **Model → Substructure → Create…** *(if your CAE version lacks this, create a **Static, General** step and convert it via the Keyword Editor — see the note at the end)*
3. Alternatively and more reliably: **Step → Create → Substructure Generation** → Continue.
4. In the dialog:
   - **Substructure identifier:** `Z101`
   - **Recovery matrix:** check **"Whole model"** or **"Region"** → choose **Region** → select `ELS_HOLES`.
     - *This is the GUI hook for stress recovery. If your version does not expose it, use the [KEYWORD EDITOR] block below.*
   - **Mass matrix:** unchecked (static only).
5. **Retained degrees of freedom:** in the same dialog (or **Model → Substructure → Retained DOF**), add:
   - `RP_BOT_SET` → DOF **1 through 6**
   - `RP_TOP_SET` → DOF **1 through 6**
   - Check **Sorted.**
6. **OK.**

Step 6 — [KEYWORD EDITOR] Add recovery + matrix output

Model → Edit Keywords → <your model name>.

Find the `*SUBSTRUCTURE GENERATE` block and make it read:

```
*SUBSTRUCTURE GENERATE, TYPE=Z101, OVERWRITE, RECOVERY MATRIX=YES, MASS
MATRIX=NO
*RETAINED NODAL DOFS, SORTED=YES
RP_BOT_SET, 1, 6
RP_TOP_SET, 1, 6
*ELEMENT RECOVERY MATRIX, ELSET=ELS_HOLES
S, E
*SUBSTRUCTURE MATRIX OUTPUT, STIFFNESS=YES, FILE NAME=perf_K, OUTPUT
FILE=USER DEFINED
```

These two lines are the ones with no GUI equivalent:

- `*ELEMENT RECOVERY MATRIX` — without it, **hole stress is unrecoverable** (§14.10).
- `*SUBSTRUCTURE MATRIX OUTPUT` — without it, **no readable** `.mtx` to parse.

OK to close the editor.

Step 7 — Job module: run generation

1. **Module: Job** → **Job → Create** → name `perf_gen` → Continue → **OK**.
2. **Job → Submit → perf_gen**.
3. **Job → Monitor** to watch for completion.
4. **Outputs:** `perf_gen.sim` (the superelement) and **`perf_K.mtx`** (the stiffness matrix — this is the file §14.9 parses).

JOB 2 (GUI) — Use the substructure (the pull test)

Step 1 — New model, import the superelement

1. **Model → Create** → name `PULL_TEST`.
2. **Model → Substructure → Import…**
3. Browse to the generated substructure library (`perf_gen`), select `Z101` → **OK**.
    - CAE creates a **substructure part** with the two retained reference nodes exposed.

Step 2 — Assembly

1. **Module: Assembly** → **Instance → Create** → select the imported substructure → **OK**.
2. The two retained RPs appear as selectable points. Create sets for them if CAE has not: `RP_BOT_SET`, `RP_TOP_SET`.

Step 3 — Step module

1. **Step → Create** → **Static, General** → Continue.
2. Time period **1.0**; **Nlgeom: Off. OK**.
    - *(A substructure is inherently linear — leave NLGEOM off.)*

Step 4 — Load module: BC on the fixed end

1. **Module: Load**
2. **BC → Create** → Step: `Step-1` → **Mechanical → Displacement/Rotation** → Continue.
3. Select **RP_BOT** → **Done.**
4. Check **U1, U2, U3, UR1, UR2, UR3 —** all = **0. OK.**
    - This grounds all six rigid-body modes.

Step 5 — Load module: the pull

Option A — prescribed displacement (recommended):

1. **BC → Create** → **Mechanical → Displacement/Rotation** → Continue.
2. Select **RP_TOP** → **Done.**
3. Set:
    - **U3 = 0.002985** (0.1% of t = 2.985 → clean linear pull)
    - **U1 = 0, U2 = 0** (no lateral drift)
    - **UR1 = UR2 = UR3 = 0** (rigid grip)
4. **OK.**
    - *To **push**, use* `U3 = -0.002985`.

Option B — applied force:

1. **Load → Create** → **Mechanical → Concentrated Force** → Continue.
2. Select **RP_TOP** → **Done. CF3 = 10000** (lb). **OK.**
3. Leave U1/U2/UR free (or restrain U1, U2 to prevent drift).

Step 6 — Field/History output

1. **Output → Field Output Requests → Create** → Continue.
2. Check **U** (Translation/Rotation) and **RF** (Reaction forces).
3. Under **State/Field/User/Time**, also request **S** and **E** so recovered stress is written.
4. **Output → History Output Requests → Create** → Domain: **Set** → `RP_TOP_SET` → request **U3, RF3.**

Step 7 — [KEYWORD EDITOR] Invoke stress recovery

Model → Edit Keywords → PULL_TEST. Inside the `*STEP` block, add:

```
*SUBSTRUCTURE RECOVERY, ELSET=ESUB
*ELEMENT OUTPUT, ELSET=ELS_HOLES
S, E, MISES
```

> Without `*SUBSTRUCTURE RECOVERY`, the recovery matrix from Job 1 exists but is **never invoked** — you will get interface reactions but **no hole stress**. This is a very easy step to skip.

Step 8 — Submit and read results

1. **Job → Create** → `perf_pull` → **Submit**.
2. **Module: Visualization** → open `perf_pull.odb`.

To read the reaction force:

- **Result → Field Output** → Variable: **RF**, component **RF3**.
- **Report → Field Output** → Position: **Unique Nodal** → Variable: **RF3** → select `RP_TOP_SET` → in the **Setup** tab check **Sum** → **Apply**.
- The written report gives the total **RF3** → `K_axial = RF3 / 0.002985`.
- *(Or: **Tools** → **Query** → **Probe values** → Nodes → click RP_TOP → read RF3.)*

To see the recovered hole stress:

- **Result → Field Output** → Variable: **S**, Invariant: **Mises**.
- Plot contours — you should see stress concentrations **at the hole edges**.
- **Sanity check (§14.8):** peak Mises at a hole edge should exceed the net-section average by roughly **2–3×**. If the field is uniform or blank, the recovery matrix did not take — go back to Job 1 Step 6.

14.11.1 GUI gotchas

- **CAE substructure support is partial.** The two Keyword Editor blocks above are not optional workarounds — they are the only way to get recovery scoping and the readable `.mtx`.
- **Keyword edits are lost if you regenerate the model** from the GUI in some workflows. If a submit silently drops your recovery, re-open **Edit Keywords** and confirm the lines

are still there.

- **Independent instance** (not Dependent) in Job 1's assembly, or you cannot mesh/select on the instance cleanly.
- **Reference points must exist in the Assembly**, not the Part, for coupling to the instance surfaces.
- `ELS_HOLES` **must be an element set**, not a geometry set, for `*ELEMENT RECOVERY MATRIX`.
- If **Model → Substructure** is missing from your CAE version, build the whole thing as a **Static** step and convert it entirely in **Edit Keywords** using the §14.2 block — the solver does not care that CAE never drew it.

14.12 CAE Keyword Corrections & Clarifications (from a real build)

Notes captured from an actual CAE-generated keyword file — the common failure points and how the lines should read.

14.12.1 Retained DOF — what they are and where they go

Retained DOF are the bridge between the substructure and the global assembly. The substructure eliminates *all* interior DOF and keeps only the retained ones — that is the entire mechanism. A few consequences people trip on:

- **A retained DOF is the opposite of a boundary condition.** A BC *removes* a DOF; a retained DOF *preserves* it through condensation. So retained DOF are **not** set in the Load module and **not** entered as a `Displacement/Rotation` BC.
- They belong to **generation** (`*Substructure Generate`), not loading.
- CAE frequently does not expose the retained-DOF dialog. If you do not see it, add the line directly in **Edit Keywords**, immediately under `*Substructure Generate`:

```
*Retained Nodal Dofs, sorted=YES
RP_BOT_SET, 1, 6
RP_TOP_SET, 1, 6
```

If this line is missing, the substructure retains nothing and the job errors or produces a useless superelement. This is the most common first-run failure.

14.12.2 Recovery scoping — two valid forms (do not use both)

To keep interior stress around the holes, recovery can be requested **two** ways:

Form 1 — shorthand `elset=` on the generate line (what CAE writes):

```
*Substructure Generate, overwrite, type=Z101, recovery matrix=YES,
elset=ELS_HOLES
*Retained Nodal Dofs, sorted=YES
RP_BOT_SET, 1, 6
RP_TOP_SET, 1, 6
```

Here `recovery matrix=YES` turns recovery on, and `elset=ELS_HOLES` scopes it to the hole-ring elements. **No separate `*Element Recovery Matrix` line is needed** — this form is self-contained.

Form 2 — long form with an explicit output selection:

```
*Substructure Generate, overwrite, type=Z101, recovery matrix=YES
*Retained Nodal Dofs, sorted=YES
RP_BOT_SET, 1, 6
RP_TOP_SET, 1, 6
*Element Recovery Matrix, elset=ELS_HOLES
S, E
```

Use one or the other, not both. Form 1 is simplest and is what CAE generates; reach for Form 2 only if you want to name specific output variables in the recovery matrix. (An earlier draft of §14.2 over-specified by combining them — Form 1 alone is correct.)

> `ELS_HOLES` **must be an** `*Elset` (element set), not an `*Nset`. `*Element Recovery Matrix` and the `elset=` shorthand both require elements.

14.12.3 `*Substructure Matrix Output` — where it goes

It lives **inside the step, after** the generate/retained/recovery lines:

```
*Step, name=Step-1, nlgeom=NO
*Substructure Generate, overwrite, type=Z101, recovery matrix=YES,
elset=ELS_HOLES
*Retained Nodal Dofs, sorted=YES
RP_BOT_SET, 1, 6
RP_TOP_SET, 1, 6
*Substructure Matrix Output, stiffness=YES, file name=perf_K, output
file=USER DEFINED
*End Step
```

This writes the readable `perf_K.mtx` (§14.9). Without it there is no ASCII stiffness matrix to parse.

14.12.4 Reference-node naming consistency

CAE may auto-name a reference point's set (e.g. `m_Set-4`) instead of the name you intended. Whatever the bottom RP set is actually called, **the coupling, retained-DOF, BC, and output lines must all use the same name.** Cleanest fix: rename it to `RP_BOT_SET` in CAE (Assembly → Set Manager → Rename) so every reference is consistent. Corrected coupling line:

```
*Coupling, constraint name=kin_coupling_bot, ref node=RP_BOT_SET,
surface=SURF_BOT
*Kinematic
```

14.12.5 Delete `*Damping Controls`

CAE may insert `*Damping Controls, structural=COMBINED, viscous=COMBINED` into the generation step. For a **static, stiffness-only** generation it is unnecessary and implies dynamic intent you do not have. Remove it.

14.12.6 The generation job produces (almost) no ODB — this is expected

A substructure *generation* job is a reduction, not an analysis. Nothing is solved, so there are no results to view. Do not expect stress, displacement, or a responsive model in the generation ODB — it will be empty or trivial, and that is **correct, not a failure.**

Generation writes instead:

- `Z101.sim` / `.sup` — the superelement (the real deliverable)
- `perf_K.mtx` — the stiffness matrix
- `.prt`, `.stt`, and library `.mtx` files

Interior stress only appears in Job 2, after the superelement is loaded and `*Substructure Recovery` is invoked (§14.7). If you submit generation and open the ODB expecting to see the tubesheet respond, you will think it failed — it did not; the superelement lives in the `.sim`/`.sup` library, not the ODB.

14.12.7 Corrected minimal generation step (all fixes applied)

```
** MATERIALS
*Material, name=cf_650
*Elastic
2.7e+07, 0.3
**
** (ELS_HOLES must be an *Elset of the hole-ring elements)
** (RP_BOT_SET, RP_TOP_SET are the two reference-node sets)
```

```
**
** STEP
*Step, name=Step-1, nlgeom=NO
*Substructure Generate, overwrite, type=Z101, recovery matrix=YES,
elset=ELS_HOLES
*Retained Nodal Dofs, sorted=YES
RP_BOT_SET, 1, 6
RP_TOP_SET, 1, 6
*Substructure Matrix Output, stiffness=YES, file name=perf_K, output
file=USER DEFINED
*End Step
```

14.12.8 Job 2 — also read the bottom (fixed-end) reaction

`RP_BOT` is fully fixed, so its reaction is the grounded reaction — read it alongside the top. It should be **equal and opposite** to the top reaction (a built-in equilibrium check).

Add both reference sets to the output, and report both:

```
*OUTPUT, HISTORY
*NODE OUTPUT, NSET=RP_TOP_SET
U3, RF3
*NODE OUTPUT, NSET=RP_BOT_SET
RF3
```

In Visualization → **Report → Field Output** → Variable **RF3**, Position **Unique Nodal**, select `RP_BOT_SET`, check **Sum** → the summed `RF3` at the bottom should equal `-RF3(top)` to within solver noise. If it does not, an interior constraint or the coupling is leaking load — investigate before trusting the stiffness.